HUGGING FACE PYTHON SCRIPT

Technical Analysis and Documentation

Project: Hugging Face Data Collection
Jira Reference: MT-1393
Document Type: Technical Analysis and Documentation
Prepared By: [Your Name]
Date: 06 November 2026

---

1. Project Overview

The Hugging Face Python script is a web-scraping application designed to collect research paper information from the Hugging Face website.

The application retrieves research paper listings for configured dates, accesses individual paper pages, extracts relevant metadata, and stores the collected information in a structured CSV file.

The application uses Python libraries such as Requests, BeautifulSoup, and Pandas to perform HTTP requests, parse webpage content, and organize the extracted information.

It also incorporates logging, error handling, retry mechanisms, and optional email notification functionality to support process monitoring and troubleshooting.

2. Purpose and Scope

2.1 Purpose

The primary purpose of the application is to automate the collection of research paper information from Hugging Face.

Instead of manually visiting individual research paper pages and collecting their details, the script performs the collection process programmatically.

The extracted information can be used for research tracking, reference purposes, and further analysis.

2.2 Scope

The analysis covers the following application functionalities:

- Retrieving research paper listings for configured dates.
- Accessing individual research paper pages.
- Extracting paper metadata.
- Identifying associated PDF links.

- Organizing collected information.
- Generating CSV output.
- Recording application activities through logging.
- Handling request failures and exceptions.
- Checking the availability of locally stored PDF files through a supporting script.

3. Application Architecture

The application follows a modular Python structure in which different files handle specific responsibilities.

The main execution logic is implemented in Main.py, while Processor.py contains the web-scraping and data-extraction methods.

Supporting modules provide configuration, logging, and email notification functionality.

3.1 Main Components

Main.py – Main Execution Controller

Responsible for coordinating the complete data collection process.

Its responsibilities include:

- Initializing the collection process.
- Retrieving the configured date range.
- Generating Hugging Face listing URLs.
- Loading listing pages.
- Extracting individual paper links.
- Calling metadata extraction methods.
- Collecting records in a data structure.
- Generating the final CSV output.

Processor.py – Scraping and Processing Module

Contains the HuggingFace_Scrape class and its associated methods.

Its responsibilities include:

- Generating date-based URLs.
- Validating URLs and HTTP responses.
- Loading webpages.
- Parsing HTML content.
- Extracting research paper links.
- Extracting paper metadata.
- Identifying PDF URLs.
- Cleaning filenames.
- Downloading PDFs when the download method is invoked.

Config.py – Configuration Module

Stores application configuration values.

The configuration includes:

- Product identification.
- SMTP server and port settings.
- Email sender and recipient settings.
- Email notification enablement.
- Logging directory settings.
- Syslog server configuration.

Sensitive configuration values, including credentials, should not be reproduced in technical documentation.

GetLogger.py – Logging Module

Contains the LogHandler functionality.

Its responsibilities include:

- Initializing the application logger.
- Creating the required log directory.
- Configuring logging.
- Recording application events and errors.
- Supporting file-based and system logging.

MailWorker.py – Email Notification Module

Contains the Mail class responsible for email notifications.

Its responsibilities include:

- Reading email configuration.
- Preparing notification messages.
- Sending process-related notifications.
- Handling email-related exceptions.

Email notifications are controlled through the EmailNotification configuration setting.

MaxvalLog.conf – Logging Configuration

Contains the logging configuration, including logger, handler, and formatter definitions.

It supports the configuration of log output and message formatting.

check_pdf_availability.py – PDF Availability Checker

A supporting utility used to compare research paper information against locally stored PDF files.

It identifies available and missing PDF documents and generates availability reports.

Helper.py – Supporting Module

The file was found in the project structure but appeared empty during the initial review. No active functionality could be established from the available inspection.

4. Technologies and Dependencies

The application uses Python and supporting libraries for web requests, HTML parsing, data processing, and validation.

The identified dependencies include:

Python: Main programming language used to implement the application.

Requests: Used to send HTTP GET requests and retrieve webpage responses.

BeautifulSoup: Used to parse HTML content and extract webpage elements.

Pandas: Used to organize extracted records into a DataFrame and export CSV files.

validator_collection: Used for URL validation through the checkers functionality.

Logging: Python logging functionality used to record application activities and errors.

SMTP: Used by the email notification module to communicate with the configured mail server.

5. End-to-End Application Workflow

The application follows a sequential data collection workflow.

5.1 Application Initialization

Main.py initializes the required components, including the logger and email notification functionality.

The application then obtains the date range through the get_date_range() method in Processor.py.

An empty list named df_data_set is initialized to store the extracted research paper records.

5.2 Date-Based Listing Retrieval

The application iterates through the generated date range.

For each date:

- The get_page_url() method generates the corresponding Hugging Face listing URL.
- A random delay is introduced before the page request.
- LoadPage() retrieves the webpage.
- The HTTP response is checked before further processing.

5.3 Research Paper Link Extraction

After successfully retrieving a listing page, Main.py calls Get_href_Papers_from_Response().

This method uses BeautifulSoup to parse the HTML response.

It identifies links containing the /papers/ path while excluding certain links, including community and date-related links.

The resulting paper URLs are collected and duplicate links are removed.

5.4 Individual Paper Processing

The application iterates through the extracted research paper URLs.

Each individual paper page is loaded using LoadPage().

After a successful HTTP response, the application invokes the metadata extraction methods.

The extracted values are stored in variables for further processing.

5.5 Metadata Extraction

The following information is extracted from individual paper pages:

- Research paper title.
- Publication date.
- Submission date.
- Abstract.
- PDF URL.
- Cleaned filename.

The application also retains the listing page URL, individual paper URL, and collection date.

5.6 Data Collection

The extracted information is appended to the df_data_set list.

Each successful record contains eight values corresponding to the final CSV structure.

5.7 CSV Generation

After the collection process, the application converts the collected records into a Pandas DataFrame.

The DataFrame is exported to:

Huggingface_Data - MT-1354.csv

The export uses index=False, preventing the DataFrame index from being added as an additional CSV column.

6. Data Extraction and Output Structure

The application extracts metadata from research paper pages and stores it in the following eight CSV columns.

Column| Description
Title| Research paper title
published| Publication date
Submitted On| Submission date
Abstract| Research paper abstract
Page URL| Hugging Face listing page for the collection date
Case Link| Individual research paper page URL
pdf link| Associated PDF URL
Date| Date currently being processed

The record is constructed in Main.py using the following logical mapping:

Title → title

published → published

Submitted On → submitted_on

Abstract → abstract

Page URL → page_url

Case Link → papers

pdf link → pdf_link

Date → date

This structure provides a consolidated record of each collected research paper.

7. HTTP Requests and Response Validation

The application uses the Requests library to retrieve webpage content.

The LoadPage() method in Processor.py is responsible for loading the requested URL.

The application includes the following validation mechanisms:

URL Validation

ValidateURL() checks whether the supplied URL is valid using the configured URL validation functionality.

HTTP Response Validation

ValidateResponse() checks the response status through response.ok.

Retry Mechanism

LoadPage() implements a maximum of three attempts.

When an exception or invalid response occurs, the application can wait for a random interval before attempting another request.

The retry delay is generated between one and five seconds.

If all attempts fail, the application records the failure and returns None.

The application also uses random delays between page-processing operations.

8. Logging and Monitoring

The application uses a centralized logging mechanism to record execution activities.

The GetLogger.py module initializes the logger, while MaxvalLog.conf provides logging configuration.

The application records information such as:

- Process initiation.
- Page loading activities.
- URL validation results.
- Successful page retrieval.
- Failed requests.
- Metadata extraction activities.

- Exception details.
- Process completion.

Log files are maintained under the configured supporting files and logs directory.

Logging helps identify execution issues and provides information for troubleshooting.

9. Email Notification Functionality

The MailWorker.py module contains the email notification implementation.

It uses the configuration values defined in Config.py to establish the email connection.

The module supports process-related notifications and error-related notifications.

The email functionality is controlled through the EmailNotification configuration setting.

The configuration reviewed during the analysis showed email notifications disabled.

Therefore, email notification functionality is implemented in the application, but its actual operation depends on the configuration.

10. PDF Handling

The application contains functionality for identifying and downloading research paper PDF documents.

10.1 PDF Link Extraction

The get_pdf_uri() method is called during individual paper processing to obtain the associated PDF URL.

The extracted PDF URL is stored in the pdf link CSV column.

10.2 PDF Download Functionality

Processor.py contains a download_pdf() method.

The method:

- Retrieves the PDF URL.
- Checks or creates the destination directory.
- Creates the destination file path.
- Retrieves the PDF response.
- Writes the response content in binary mode when the response is successful.

However, the PDF download invocation in Main.py appeared commented out during the code review.

Therefore, PDF URL extraction is part of the active collection workflow, while automatic PDF downloading through that invocation should be considered inactive in the reviewed version.

10.3 PDF Availability Checker

The separate check_pdf_availability.py script checks whether expected PDF documents are available in the configured local directory.

It reads the CSV metadata and compares paper titles and filenames against locally stored PDF documents.

The utility supports:

- Filename normalization.
- Title matching.
- Identification of available PDFs.
- Identification of missing PDFs.
- Reverse checking of additional PDF files.
- Date-wise availability reporting.

The identified output files include:

- pdf_availability_report.csv
- pdf_missing_by_date_report.csv

This utility is separate from the primary Main.py execution workflow.

11. Error Handling and Failure Tracking

The application contains exception-handling mechanisms to manage different failure scenarios.

The identified scenarios include:

Invalid URLs: URL validation is performed before sending requests.

Failed HTTP Responses: Response validation is used to identify unsuccessful page retrieval.

Request Exceptions: Exceptions during page loading are logged.

Retry Exhaustion: Failed attempts are recorded after the configured retry limit is reached.

Permission Errors: Main.py includes handling for file-system permission-related errors.

Network Connection Failures: The application includes a network-related error message advising VPN connectivity.

Quota and Timeout Errors: Specific exception handling is included for quota-exceeded and read-timeout conditions.

The application also maintains separate lists for failed listing-page URLs and failed individual paper URLs.

These lists help track unsuccessful collection operations.

12. Hugging Face Interaction and Model Analysis

The reviewed implementation interacts with Hugging Face through its website pages.

The application retrieves research paper listing pages and individual paper pages, then extracts information from their HTML content.

The reviewed code uses HTTP requests and HTML parsing for this purpose.

No model inference, model training, or direct Hugging Face Transformers model interaction was identified in the reviewed execution flow.

Although the extracted research paper abstracts may contain terms such as transformers, machine learning, or artificial intelligence, these are collected as research-paper content and do not indicate that the application itself executes those models.

The identified Hugging Face interaction is therefore primarily website-based research paper data collection.

13. Technical Observations and Limitations

The following observations were identified during the code review.

1. The collection date range is configured within the application and should be verified before execution.

2. The application depends on successful network access to retrieve Hugging Face webpages.

3. The retry mechanism is limited to three attempts.

4. The PDF download method exists, but its invocation in the main workflow appeared commented out.

5. Email notification functionality depends on the configuration setting and was disabled in the reviewed configuration.

6. The PDF availability checker operates as a separate supporting utility.

7. The application uses HTML parsing, so changes to the Hugging Face webpage structure may affect extraction results.

8. The reviewed request calls use requests.get(). A request timeout was not identified in the visible code sections.

9. Failed URLs are collected in separate lists. The reviewed portions should be checked further to establish whether these lists are exported into a separate report.

10. The complete runtime behaviour has not been established through an end-to-end execution test as part of this code-review analysis.

These observations are documented to distinguish implemented functionality from configuration-dependent or unverified behaviour.

14. Overall Conclusion

The Hugging Face Python application is a modular web-scraping solution designed to collect research paper information from Hugging Face and store the extracted metadata in a structured CSV file.

Main.py coordinates the overall workflow, while Processor.py handles webpage retrieval, validation, HTML parsing, metadata extraction, and PDF-related functionality.

Supporting modules provide configuration management, logging, and email notification capabilities.

The application also includes a separate PDF availability checker to compare collected paper information with locally available PDF files.

The analysis establishes that the primary purpose of the reviewed implementation is automated research paper metadata collection rather than machine learning model training or inference.

This documentation summarizes the existing application architecture, processing workflow, data structure, supporting functionalities, and technical observations based on the source-code review.

End of Document